FINANCE / PS-1

TRACEGUARD

Financial Fraud & Transaction Risk Detection and Investigation Workbench



A research-backed implementation blueprint for
Srujana Tech Fest 2026 - 48-hour software hackathon

Includes current Indian threat context, open-source review, architecture, data and modelling guidance, validation protocol, API/UI plan, build schedule, judge Q&A and AI coding handoff.

Research cut-off: 24 October 2026

Version 1.0 | Team implementation handoff

Prepared from the supplied PS-1 statement and participant rubric, plus linked public sources.

How to use this report

This document is an implementation blueprint for Finance problem statement F-01, “Financial Fraud and Transaction Risk Detection.” It is designed to be usable by a student team or a coding AI: it provides requirements, an architecture, data decisions, testing rules, acceptance checks, a schedule and a linked source pack.

> **Evidence discipline**
> Facts attributed to external sources are cited with clickable source IDs such as **[S05]**. Proposed features, priorities, thresholds, schemas and architecture are team-design recommendations, not facts claimed by the cited sources. Do not describe synthetic benchmark performance as Indian banking or UPI performance.

Scope boundaries

• **In scope:** retrospective transaction-risk scoring, account-linked sequence analysis, case grouping, evidence-backed explanations, review queue, evaluation and deterministic demonstration fixtures.

• **Out of scope:** real-time bank integration, actual UPI authorisation, use of private bank data, access to I4C Suspect Registry, real payment blocking, criminal attribution and production deployment.

• **As-of date:** 24 October 2026. Current statistics and advisories can change; the linked original source is authoritative for updates.

What the final submission must prove

A judge must be able to load or replay a scenario, see the system generate a case, inspect the supporting transaction chain, understand the score components, review a legitimate high-volume case that should not be automatically labelled as fraud, and inspect a measured evaluation result. If one of these paths is not implemented, mark it as future work rather than implying it works.

Contents

How to use this report . . . 2
Scope boundaries . . . 2
What the final submission must prove . . . 2

Contents . . . 3

1. Executive decision . . . 6
Core thesis . . . 6
Five non-negotiable success criteria . . . 6
The three decisions to make immediately . . . 6

2. Problem statement and judging rubric . . . 6
Srujana rubric: fixed weights . . . 7
Hackathon format that shapes the build . . . 7

3. Current Indian context: what is verified . . . 7
UPI volume establishes scale, not fraud prevalence . . . 7
I4C–RBIH focus on mule-account intelligence . . . 7
Very recent adjacent threat: abuse of payment APIs . . . 8
Research trend: temporal drift and unequal error costs . . . 8
Responsible AI reference . . . 8

4. Product definition and differentiation . . . 8
Target user and job-to-be-done . . . 8
Product promise . . . 8
Differentiators: each must be functional . . . 9
What not to claim . . . 9

5. Open-source landscape and reuse strategy . . . 9
How to study existing repositories efficiently . . . 9

6. Functional specification and scope control . . . 10
Must-have (P0) - finish these before adding extras . . . 10
Should-have (P1) - only after P0 passes . . . 10
Avoid unless all previous work is stable . . . 10
User story / use-case flow . . . 10

7. Architecture and implementation choices . . . 11
Logical component flow . . . 11
Recommended API contract . . . 11
Design choices that reduce risk . . . 11

8. Data plan and provenance . . . 12
Primary benchmark: PaySim . . . 12
How to retrieve and validate data . . . 12
Field policy . . . 12
Data contract for the app . . . 12
Privacy and data minimisation . . . 13

9. Feature engineering, rules and graph design . . . 13
Point-in-time feature building . . . 13
Three rule families to implement . . . 13
Graph model: keep it small and explainable . . . 14

10. Scoring, prioritisation and explanations . . . 14
Keep evidence components separate . . . 14
Priority policy for the prototype . . . 14
Evidence schema . . . 14
Use SHAP with restraint . . . 14

11. Machine-learning plan . . . 15
Baseline then one stronger model . . . 15
Training/evaluation split . . . 15
Metrics for a rare-event task . . . 15
Model acceptance gates . . . 16

12. Scenario catalogue and test plan . . . 16
Deterministic demo scenarios . . . 16
Testing layers . . . 16
Definition of done . . . 17

13. Interface and endpoint details . . . 17
Screen A: investigation queue . . . 17
Screen B: case investigation . . . 17
Screen C: evaluation . . . 17
Error handling and edge cases . . . 17

14. Security, governance and ethical boundaries . . . 18
Minimum safeguards . . . 18
RBI FREE-AI principles translated into build tasks . . . 18
Related incident response links . . . 18

15. Remaining hackathon schedule (24-26 October) . . . 18
Checkpoint strategy . . . 19

Team role allocation . . . 19

16. One-minute demo and likely judge questions . . . 19

Recommended 60-second script . . . 19

Likely questions and defensible answers . . . 20

17. Coding-AI handoff: implementation prompt . . . 20

A useful source inspection sequence for a coding AI . . . 21

Repository acceptance checklist . . . 21

18. Final go/no-go checklist . . . 22

Final build order . . . 22

19. Verified source library . . . 23

1. Executive decision

Build **TraceGuard**: an explainable transaction-risk and investigation workbench. Its distinct value should be a compact, connected-event investigation workflow—not a claim that it has invented fraud detection or achieved bank-grade detection performance.

> **Recommended positioning**
> "TraceGuard turns suspicious transaction signals into reviewable cases: it links related events, shows the evidence behind each alert, makes missing history visible, and helps an investigator prioritise review. The prototype is evaluated on public/synthetic data and does not connect to banks."

Core thesis

Fraud models can score a transaction, but the problem statement also calls for considering sequences, account relationships, unusual behavioural change, explainable risk levels and investigation. The product should make that complete workflow the centre of the demo.

Five non-negotiable success criteria

1. **End-to-end:** data ingestion or scenario replay -> feature calculation -> scoring -> case creation -> investigator review -> status/audit record.
2. **Independent evidence:** model output, explicit rules and graph/sequence signals remain separately visible; do not merge them into an unexplained black-box number.
3. **Time-correct features:** each feature only uses information available at the declared scoring time. No target label or existing fraud flag is used as a model input.
4. **Honest evaluation:** chronological holdout; precision/recall-focused metrics; no "accuracy" headline alone; reproducible commands and dataset provenance.
5. **Robust demo:** suspicious chain, legitimate burst, and insufficient-history case all work locally without network dependency.

The three decisions to make immediately

• Use PaySim as the baseline public synthetic dataset if it can be downloaded and validated quickly. Keep a smaller, fully documented, hand-verified synthetic stream for deterministic demo scenarios. **[S07] [S08] [S09]**

• Use FastAPI + scikit-learn/XGBoost + NetworkX + SQLite/PostgreSQL + React/Vite unless the team has a stronger, already-working stack. This is a practical recommendation, not an official requirement.

• Build the working investigation screen first; do not wait for the final model before connecting the UI to the backend.

2. Problem statement and judging rubric

F-01 asks for a solution that analyses transaction activity to identify, assess and prioritise activity that may warrant investigation. The statement explicitly mentions amount, frequency, timing, location, transaction history, account relationships, spending/transfer changes, false alerts, changing behaviour and incomplete information. The expected outcome is an interpretable prototype that supports case investigation while balancing detection with unnecessary alerts.

The wording above is a faithful summary of the supplied F-01 statement; use the original problem statement in the team submission for exact wording.

Srujana rubric: fixed weights

Criterion	Weight	Evidence the team should show
Problem understanding and relevance	15	User, problem scope, assumptions and relevance explained clearly.
Technical depth and implementation	20	A real feature/model pipeline, design choices and integration explained.
Functional completeness and reliability	15	A coherent end-to-end workflow runs reliably with representative data.
Domain analysis and correctness	20	Correct calculations, sound assumptions, explainable outputs and clear boundaries.
Validation and robustness	10	Normal cases, edge cases, incomplete data and failure cases tested.
User experience and accessibility	10	A judge/user can find the main task and understand alerts.
Originality and presentation	10	A thoughtful distinction and clear demonstration, not a technology list.

This is drawn from the uploaded “Srujana 2026 Participant Rubric and Submission Guide” (pages 1–3). The guide permits public, synthetic or team-generated data and warns that additional features do not automatically mean a better project. It also says judges may change inputs, test scenarios and ask about edge cases. Treat that as a build requirement, not a presentation detail.

Hackathon format that shapes the build

The official event page says the 48-hour hackathon starts 24 October 2026 at 09:30 IST; teams contain 2–4 people; the one-minute working demo is the main basis for first-round shortlisting; and shortlisted teams are evaluated on the working implementation, repository and documentation. The submitted repo name must follow the pattern **Srujana_hackathon2026_projectname**. [S01]

> **Execution implication**
> The first checkpoint is a running slice of the real product, not a model notebook. Every eight-hour evaluation window should produce a visible, testable increment.

3. Current Indian context: what is verified

UPI volume establishes scale, not fraud prevalence

At research time, NPCI’s official statistics page displayed 24,068.38 million UPI transactions with a value of ₹29,37,396.67 crore for September 2026. This is roughly 24.07 billion transactions and ₹29.37 lakh crore in value. These figures establish the scale of the payment system; they are not a fraud rate and must not be presented as such. The NPCI page is dynamic and may update. [S06]

I4C–RBIH focus on mule-account intelligence

On 12 May 2026, the Ministry of Home Affairs reported that the Indian Cyber Crime Coordination Centre (I4C) and Reserve Bank Innovation Hub (RBIH) signed an MoU to strengthen information sharing and AI-driven detection of mule accounts and cyber-enabled financial fraud. The release describes feeding suspect-registry information into AI-driven fraud-detection systems. [S03]

A separate Ministry release posted 11 August 2026 reports that, as of 30 June 2026, CFCFRMS had saved more than ₹11,158 crore in more than 32.80 lakh complaints; it separately reports 32.08 lakh Layer 1 mule accounts shared through the Suspect Registry and transactions worth ₹25,698 crore declined. These are separate metrics with different meanings. Do not combine them, and do not imply that they measure a student product’s effect. [S04]

Very recent adjacent threat: abuse of payment APIs

CERT-In Advisory CIAD-2026-0046 was issued on 9 October 2026 with a Critical severity rating. It describes campaigns targeting NBFCs, digital lenders, fintechs, payment service providers and wallet operators. The advisory includes compromised exposed applications/APIs, stolen credentials, direct invocation of disbursement/fund-transfer APIs, dummy references, rapid multiple transfers and credits to mule accounts. It recommends controls including MFA, API inventories, per-session authentication, validation of payout records, velocity limits and real-time monitoring. **[S05]**

Critical scope distinction

This advisory describes a mix of cybersecurity/control-plane compromise and suspicious transfers. Transaction history alone may not reveal how a privileged API or credential was compromised. TraceGuard may use a seeded “unmatched reference + rapid payout” demo as an adjacent scenario, but the report must not claim to detect API compromise without application/security telemetry.

Research trend: temporal drift and unequal error costs

Open-access research published on 18 June 2026 describes financial fraud detection as combining extreme class imbalance, temporal non-stationarity and a sharp cost gap between missed fraud and false alarms. A separate September 2026 article discusses evolving anomalies and concept drift in transaction streams. These works support testing over time and treating the threshold as an operating decision. Their datasets and reported findings must not be assumed to transfer directly to Indian UPI. **[S14] [S15]**

Responsible AI reference

The RBI’s August 2025 FREE-AI Committee report sets out principles including trust, people first, fairness and equity, accountability, explainability, and safety/resilience. For TraceGuard, translate those into visible evidence, analyst review, data minimisation, model limitations and reproducible testing. This is a design reference, not a claim that this college prototype is a regulated financial product. **[S20]**

4. Product definition and differentiation

Target user and job-to-be-done

Primary user: a fraud analyst/investigator reviewing an alert queue. **Job:** decide which case to inspect first, understand why it was flagged, inspect related events and record a review outcome. The demo uses a synthetic analyst workspace; it does not claim to integrate into a bank.

Product promise

One-sentence pitch

TraceGuard groups suspicious transaction sequences into explainable cases, surfaces linked accounts and supporting evidence, and helps a human investigator prioritise review without treating every unusual transaction as fraud.

Differentiators: each must be functional

Differentiator	Visible behaviour in the prototype	How to verify it
Sequence-aware cases	Case contains multiple related transaction events, not just a single red row.	Open the seeded chain and inspect ordered events.
Independent evidence	Model score, rule signals and network context remain separate.	Toggle/show each component; inspect evidence values.
Uncertainty and coverage	Insufficient history is visible and not silently treated as suspicious.	Replay a new account with only one or two events.
Legitimate-burst challenge	High activity can be normal for merchant/payroll behaviour.	Replay a benign burst and inspect why it is not auto-labelled fraud.
Replayable, auditable case	Case ID, scenario ID, feature version, score components and analyst status are recorded.	Reload/replay and confirm determinism.
Time-aware validation	Metrics reflect a chronologically held-out sample.	README shows split boundaries and one command to reproduce evaluation.

What not to claim

• Do not claim “real-time UPI fraud detection” if the app only replays CSV data.

• Do not call an uncalibrated model output a probability of guilt or a probability that a transaction is fraudulent.

• Do not claim financial loss prevented, bank integration, I4C/Suspect Registry access, or regulatory approval.

• Do not describe generic rule/graph/SHAP features as original by themselves. Existing repositories already implement graph-based PaySim examples and UPI-labelled app demos. **[S11] [S12] [S13]**

5. Open-source landscape and reuse strategy

This is a targeted reconnaissance of public repositories, not an exhaustive census of GitHub. Repository existence establishes prior art, not quality, security, correctness or the number of competing teams at Srujana.

Reference	What the repository/source says	Implication for TraceGuard
IBM AMLSim **[S10]**	Synthetic banking transactions with known laundering patterns for ML/graph evaluation. Apache-2.0 listed; README describes older Java/Python/dependency versions.	Use as research/reference if time permits; do not make legacy setup a blocker. Verify current licence and all dependency licences before copying code.
Neo4j graph fraud example **[S11]**	PaySim + Neo4j Graph Data Science/Bloom. README says example targets older Neo4j versions.	Graph detection already exists. Borrow concepts, not a dependency stack you cannot finish.
Neo4j PaySim ML demo **[S12]**	Graph-native ML demo for money-mule detection on synthetic PaySim; requires an appropriate Neo4j/Aura environment per README.	Use it to understand graph-native AML; avoid expensive setup if NetworkX is adequate.
Public UPI ML project **[S13]**	Full-stack app/notebook presented as UPI fraud detection. Its landing page does not prove the underlying dataset is real Indian UPI data.	Treat UPI in the title as a project claim, not data provenance evidence.
PaySim paper and benchmark **[S07] [S08]**	Synthetic mobile-money data designed because real financial transaction datasets are difficult to share.	Good reproducible baseline, but not a substitute for real Indian transactions.

How to study existing repositories efficiently

1. Read the README and record the claimed task, data origin, split method, metrics and runtime dependencies.
2. Inspect the notebook/source to check label leakage, train/test splitting, metric choice and whether features exist at decision time.
3. Check the LICENSE file in the default branch and licences for datasets/models/assets. A public GitHub repository is not automatically permissively licensed.
4. Run it only if setup time is bounded; do not make TraceGuard dependent on Neo4j Enterprise, a paid hosted API or a heavy AML simulator.

5 Record which ideas were informed by existing work and which code was actually reused; follow Srujana's rules on attribution and originality.

> **Do not overstate the novelty**
> The defensible differentiator is the quality of the complete, testable workflow: sequence-level cases, evidence traceability, history coverage, legitimate-control scenarios and honest evaluation. Graphs, XGBoost and SHAP are established tools, not novelty claims.

6. Functional specification and scope control

Must-have (P0) - finish these before adding extras

ID	Requirement	Acceptance condition
P0-1	Load seeded transactions or a validated CSV.	Visible row count; schema errors explained; duplicate IDs handled or rejected.
P0-2	Score each event using reproducible rules.	Same input and configuration produce same result.
P0-3	Create a case from a suspicious sequence.	Case lists its contributing transactions and rule/graph evidence.
P0-4	Investigator queue and detail view.	Queue sorts by documented triage policy; case can be opened and reviewed.
P0-5	Three scenario fixtures.	Suspicious chain, legitimate burst and insufficient-history replay work offline.
P0-6	Baseline ML comparison.	Run training/evaluation command; model artifact includes feature list and split.
P0-7	Metrics and limitations page.	At minimum AP/PR curve, precision, recall, false positives and top-K result on holdout.
P0-8	Tests and documentation.	Fresh clone can follow documented startup; tests pass; data provenance is documented.

Should-have (P1) - only after P0 passes

• Feature attribution for the tree model with SHAP if installation is stable; show direction and values carefully. **[S19]**

• Case export to JSON/CSV or a concise printable report.

• Analyst feedback field with statuses such as Open, Reviewed, Needs More Evidence, Closed - Not Suspicious and Escalate for Review. These are prototype workflow labels, not legal determinations.

• Deterministic "what changed?" replay that modifies a seeded event amount/timing and displays the resulting components.

Avoid unless all previous work is stable

• Graph neural networks, LLM-based fraud reasoning, external alert integrations, live financial APIs, sophisticated user roles, multi-tenant cloud deployment and automatic blocking.

User story / use-case flow

1 Analyst opens TraceGuard and selects a locally loaded scenario or imports a valid CSV.

2 The ingestion module validates schema, transaction uniqueness, type/amount and temporal order.

3 Feature builder computes point-in-time behavioural summaries and graph/sequence signals.

4 Rules and optional ML model produce distinct evidence outputs.

5 Case grouper groups related events and generates a reasoned priority status.

6 Analyst opens a case, inspects timeline and graph, marks a review status and adds notes.

7 Evaluation screen shows benchmark metrics and test limitations separately from the live demo.

7. Architecture and implementation choices

A small modular architecture will be easier to test than a distributed microservice design. A monolith with clear modules is appropriate for a 48-hour student build.

Layer	Suggested implementation	Responsibilities
Frontend	React + Vite; Cytoscape.js or a React network visualisation.	Queue, case detail, network, timeline, scenario replay and metrics.
API	FastAPI (Python).	Input validation, scenario endpoints, score endpoints, cases and metrics.
Feature logic	Python + pandas/Polars as needed.	Schema checking and causal/point-in-time-safe feature calculation.
Models	scikit-learn baseline; optional XGBoost/LightGBM.	Baseline and supervised ranking model; fixed model artifact.
Graph analysis	NetworkX initially.	Small graph features, connected-case grouping and reproducible paths.
Persistence	SQLite for minimum scope; PostgreSQL only if already familiar.	Store scenarios/cases/review state and audit events.
Evaluation	scikit-learn metrics.	Time split, AP/PR, precision/recall, top-K and confusion counts.

Logical component flow

1. Input	2. Analyse	3. Case workflow	4. Human review
CSV / seeded event stream; schema checks.	Feature builder -> rules + ML + network features.	Evidence aggregation -> queue -> case graph/timeline.	Reviewer sees factors, coverage, notes and status.

Recommended API contract

Method and path	Purpose	Response expectation
GET /health	Readiness check.	Returns service status and model/demo mode.
POST /api/transactions/import	Validate/import CSV.	Returns accepted/rejected row counts and field-level errors.
GET /api/cases	List cases with filters.	Returns case ID, priority, status, signal summary and coverage.
GET /api/cases/{case_id}	Inspect one case.	Returns transactions, graph edges, evidence, score components and review history.
POST /api/scenarios/{id}/replay	Run a seeded deterministic scenario.	Returns resulting events, signals, case and expected test assertions.
PATCH /api/cases/{case_id}/review	Record a reviewer action.	Stores status, note, reviewer label and audit timestamp.
GET /api/evaluation/summary	Show model evaluation.	Returns metrics, split boundaries, sample counts and dataset disclosure.

Design choices that reduce risk

• Keep feature functions pure where possible: same input history and event -> same feature output.

• Keep inference routes separate from benchmark evaluation routes. Never expose the label field as an input to inference.

• Have an explicit DEMO mode that loads local fixtures. Make the UI visibly identify demo/synthetic data.

• Persist only the minimum needed for the prototype. Use synthetic account IDs; never accept real bank credentials.

• Use a small `case_evidence` record to explain signals. Do not require an LLM to reconstruct explanations from opaque JSON.

8. Data plan and provenance

Primary benchmark: PaySim

The commonly referenced PaySim1 file contains 6,362,620 transactions, of which 8,213 are labelled fraudulent (about 0.129%). The dataset spans a simulated 30-day period; the `step` variable is an hourly simulation step. The 11 common fields are `step`, `type`, `amount`, `nameOrig`, `oldbalanceOrg`, `newbalanceOrig`, `nameDest`, `oldbalanceDest`, `newbalanceDest`, `isFraud` and `isFlaggedFraud`. **[S07] [S08]**

> **What these numbers mean**
> PaySim is synthetic mobile-money data created because real financial records are difficult to share. It is useful for reproducibility and method development. It is not an authentic Indian UPI dataset and its score is not a claim of bank-ready performance. **[S07] [S08]**

How to retrieve and validate data

1. Open the linked PaySim dataset page **[S09]** and check access, current licence/terms, file size and available versions before downloading.
2. Store the unmodified source file outside Git or in an explicitly ignored local directory. Do not check the full multi-million-row CSV into the repository unless the rules and repository limits permit it.
3. Record the dataset source URL, download date, filename, byte size, row count, column names and a SHA-256 hash in `docs/data_provenance.md`.
4. Run validation: required columns, null counts, type domains, non-negative amounts, valid labels, monotonic time ordering after sort, duplicate rows/IDs and label prevalence.
5. Use a reproducible, time-ordered sample for development if runtime or memory is constrained. Never describe a downsampled or label-balanced test set as representative without disclosing its construction.

Field policy

Field/category	Use in MVP	Reason / guardrail
step, type, amount	Include.	Available in the benchmark; document that `step` is hourly.
nameOrig, nameDest	Use for grouping/graph only; encode categorical identity only if justified.	IDs are not stable user semantics; avoid pretending an ID itself implies guilt.
oldbalance/newbalance fields	Exclude from the first model; evaluate only in an explicit ablation if justified.	Can encourage simulator-specific shortcuts; ensure feature availability matches the declared scoring time.
isFraud	Target label only, never a feature.	Direct target leakage.
isFlaggedFraud	Exclude from prediction features.	An existing flag/rule-like field may give the model an unfair shortcut; document any separate analysis.
channel, location, device, recipient age	Not present in the base PaySim schema.	Only use in separate synthetic fixtures and label them as demo-only fields.

Data contract for the app

Define a canonical normalized event independent of PaySim field names. Suggested required fields: `transaction_id`, `event_time` (or `event_step` where the source is step-based), `source_account_id`, `destination_account_id`, `amount`, `transaction_type`, and `source_dataset`. Optional fields: `channel`, `location_bucket`, `event_status`, and `metadata` only if a dataset actually provides them. Keep labels in a separate evaluation-only table keyed by transaction ID.

Privacy and data minimisation

Use synthetic account identifiers and avoid personal names, real UPI IDs, phone numbers, bank account numbers, credentials or account exports. If a teammate tests with any real information accidentally, remove it from the demo dataset, repository and logs. Do not connect to a personal bank account to make the prototype look more realistic.

9. Feature engineering, rules and graph design

Point-in-time feature building

Declare the prototype as **post-event monitoring**: a transaction has been recorded and is being analysed for investigation. For each event, features may use its own amount/type/IDs and prior events up to that point. No later transaction, label, future account aggregate or information unavailable in the declared mode may influence the score. If the team later implements pre-authorization scoring, rebuild the feature policy to exclude any post-transaction fields.

Feature	Definition sketch	Caution
source_count_1h / 24h	Number of prior outgoing transactions in past time window.	PaySim uses hourly steps; do not claim minute-level resolution.
source_amount_sum_24h	Sum of prior outgoing value in a trailing window.	Use only rows before current event.
recipient_new_to_source	Whether source has previously paid this recipient.	Needs an accurate per-account prior history.
amount_vs_history	Current amount versus robust statistic (median/IQR or percentile) of past source transactions.	Use “insufficient history” instead of zero or extreme fallback.
incoming_degree / outgoing_degree	Unique counterparties in trailing history.	Bound the window; large graphs can become costly.
pass_through_ratio	Amount sent onward soon after receipts divided by incoming amount in a defined window.	Not evidence of fraud on its own; define treatment for netting, reversals and zero denominators.
shared_destination_signal	Whether a case shares a destination with other suspicious cases in a bounded time window.	A shared merchant/collector may be legitimate; require context and display evidence.
history_coverage	Count/age of prior observations and missing required fields.	Use as a caveat/eligibility gate, not an automatic fraud signal.

Three rule families to implement

Rule R1: rapid pass-through

Flag a review signal when an account receives one or more transfers and sends a meaningful share onward within a configured time window. Parameters (window length, proportion, minimum amount) are demo configuration values selected from validation scenarios, not universal fraud thresholds. Evidence should include the inbound/outbound event IDs, time gaps and computed ratio.

Rule R2: fan-in/fan-out or connected destination

Identify a short sequence in which multiple sources pay an account and that account disperses funds to multiple destinations, or multiple suspicious cases converge on a destination. Evidence should show the nodes, edges, time span and event IDs. Do not imply that graph shape alone proves laundering.

Rule R3: sudden behaviour shift

Compare recent frequency/amount with the account’s earlier baseline. Require enough history and define robust statistics. If a new merchant, payroll processor or high-volume account has a legitimate reason, the system should surface the signal as a reason to review, not as a definitive verdict.

Graph model: keep it small and explainable

Use a directed graph where accounts are nodes and transactions are timestamped, amount-carrying edges. For the MVP, NetworkX is enough for a modest, seeded graph. Compute bounded-window degrees, simple two- to three-hop paths and candidate shared destinations. Avoid whole-history graph traversal on each page render; precompute or cache graph features per scenario.

> **Graph acceptance test**
> The suspicious-chain fixture must highlight a connected path and list the exact transactions forming it. The benign-burst fixture must demonstrate a high-volume account whose activity is not automatically treated as fraud solely because it has many edges.

10. Scoring, prioritisation and explanations

Keep evidence components separate

For each transaction and case, display separate fields: supervised model score; triggered rule IDs and measured values; graph/sequence signals; history coverage; and the final review priority. This prevents users from mistaking a ranking score for a calibrated probability.

Priority policy for the prototype

Do not hard-code an unvalidated weighted sum and then label it a probability. For the first version, implement a deterministic, documented triage policy whose thresholds are selected on a validation split or explicitly marked as scenario settings. Example policy structure:

• **High review priority:** strong network/sequence evidence plus a model/rule signal above a validation-selected threshold, or a seeded pattern explicitly designed as high-priority in the demo fixture.

• **Medium review priority:** one substantial independent signal or an unusual model score without corroborating graph context.

• **Low / monitor:** weak anomaly with little supporting evidence.

• **Insufficient history:** a separate coverage status that prevents the case from appearing “low-risk” just because the model lacks data, and prevents missingness from itself meaning fraud.

These labels and policy examples are design suggestions. Adjust them after validation; do not report an operationally approved fraud threshold.

Evidence schema

Evidence item	Example contents
Signal ID	R1_RAPID_PASS_THROUGH, GRAPH_FAN_OUT, AMOUNT_DEVIATION.
Human explanation	“Account A received 3 transfers and sent 82% of the combined amount onward within the configured window.”
Computed values	Inbound event IDs, outbound event IDs, amount sum, ratio, time gaps, window and threshold used.
Source provenance	Dataset/scenario ID, source row IDs, feature code version and run ID.
Limitations	History coverage, missing fields, synthetic scenario flag and any assumptions.

Use SHAP with restraint

SHAP TreeExplainer can attribute components of an ensemble-tree model output to features. It does not establish causality, criminal intent or factual truth. The actual output space matters: depending on the model/configuration, values may sum to a raw margin/log-odds rather than a user-facing probability. See the official TreeExplainer documentation. **[S19]**

Recommended UI wording: “Model score elevated mainly by recent transaction-count deviation and unfamiliar recipient.” Avoid: “SHAP proves account is fraudulent.” For each explanation, show relevant raw values and permit the reviewer to open the contributing events.

11. Machine-learning plan

Baseline then one stronger model

Stage	Implementation	Purpose / go-no-go rule
A. Rule baseline	Three deterministic rules on seeded and benchmark cases where fields exist.	Provides a transparent baseline and a working fallback.
B. Simple ML	Logistic regression or RandomForest/HistGradientBoosting in scikit-learn.	Confirms that feature pipeline and metrics run end to end.
C. Tree boosting	XGBoost or LightGBM only if environment/setup is reliable.	Potentially stronger tabular model; compare on the same time split.
D. Explainability	Model feature importance/SHAP if time and versions allow.	Adds local explanations without replacing rule evidence.

XGBoost documents `scale_pos_weight` as one option for imbalance; the documentation describes a common starting ratio of negatives to positives. This is a parameter to tune and evaluate, not a magic fix. **[S18]**

Training/evaluation split

For the 744 hourly steps described for common PaySim1, a simple proposed chronological split is steps 1–520 for training, 521–632 for validation and 633–744 for final test. These boundaries are a design choice, not a standard. Inspect label distribution in each segment before training; if the positive count is too low in a segment, choose and document another chronological split. The final test must remain untouched until final evaluation.

Feature engineering must be replayed sequentially: the account history used for a validation/test event may contain earlier events from the permitted history, but may not contain later rows. Any imputer, encoder, scaler, feature selector or threshold choice is learned using training/validation data only. scikit-learn describes test-data leakage and recommends fitting preprocessing only on training data. **[S17]**

Metrics for a rare-event task

Metric	Why it belongs	Reporting rule
Average Precision (AP)	Summarises precision/recall ranking quality.	Report exact split and class prevalence. **[S16]**
Precision / Recall	Trade-off between alert correctness and detected fraud labels.	Report at the selected validation threshold.
False positives	Shows legitimate transactions/cases incorrectly surfaced.	Report counts and rate; distinguish transaction-level from case-level.
Precision@K	Reflects investigator capacity to examine top K alerts.	Set K explicitly, e.g. 20 or 50 based on the demo queue; do not generalise beyond benchmark.
Confusion matrix	Supports clear interpretation.	Include TP, FP, FN, TN counts and threshold.
Calibration (optional)	Checks whether stated probabilities correspond to empirical frequencies.	Only use calibrated-probability wording if calibration is measured on held-out data.

Accuracy trap

If a benchmark has approximately 0.129% positive labels, a model that predicts every row as legitimate can be about 99.87% accurate while detecting zero positives. This is why accuracy cannot be the headline metric for this dataset. Values follow from the commonly reported PaySim counts in [S08].

Model acceptance gates

• Do not call model B/C “better” unless it beats the baseline on the same held-out split and the metrics relevant to the intended alert budget.

• Do not tune the final threshold on the final test set.

• Report the minority label count in train, validation and test.

• Run an ablation without graph features and, where applicable, without balance columns to test whether gains are meaningful or simulator-specific.

• If time is short, a working rule + graph investigation system is preferable to an unvalidated ML model presented as production-grade.

12. Scenario catalogue and test plan

Deterministic demo scenarios

ID	Fixture	Expected behaviour	What it proves
D01	Rapid pass-through across 3–5 seeded accounts.	Create a case; show ordered path and the exact transfers that support the rule.	Connected sequence reasoning.
D02	Legitimate merchant/payroll burst with repeated small payments.	Show high activity but do not label fraud solely from volume; explanation mentions regular pattern.	False-positive awareness.
D03	New account with minimal history.	Show limited-history state; avoid overconfident behavioural comparison.	Uncertainty handling.
D04	One anomalous high-value transfer without connected evidence.	Elevate for review based on its signals, but do not assert guilt.	A single unusual transaction is not automatically fraud.
D05	Missing amount/time/account field or duplicate transaction ID.	Reject with field-level error or handle via documented duplicate policy.	Input robustness.
D06	Replay D01 after changing time gap/amount.	Recompute evidence and priority deterministically.	Working interaction, not a static mock-up.
D07	Existing `isFlaggedFraud` / target label columns supplied to an import.	Exclude/reject as inference features; labels restricted to offline evaluation pipeline.	Leakage defence.

Testing layers

1. **Unit tests:** amount/time validation, rolling aggregates, first-time beneficiary, pass-through ratio edge cases, graph paths and missing-history status.
2. **Integration tests:** import -> feature builder -> score -> case -> API response -> review status.
3. **Model tests:** deterministic seed, same split, saved feature names, no target/flag columns, metric computation and split counts.
4. **UI tests:** loading/error/empty states; queue filtering; case details; scenario rerun; screen-reader-friendly labels where feasible.
5. **Demo rehearsal:** clear local data, restart app, run each scenario with Wi-Fi disabled if no external dependencies are needed.

Definition of done

Check	Pass condition
Fresh start	Documented command launches backend and frontend without manual edits.
Data import	Valid sample accepted; invalid sample produces clear row/field errors.
Scenario replay	D01, D02 and D03 produce expected, inspectable outputs.
Evidence	Every case reason links back to source event IDs and measured values.
Evaluation	Metrics are reproducible, chronological split is documented, and no test-set tuning.
Repo quality	README, dependency list, data provenance, model card, tests and limitations included.
Claims	No unsupported claims of real UPI data, fraud prevented, API access or production readiness.

13. Interface and endpoint details

Screen A: investigation queue

• Columns: case ID, review priority, earliest/latest event time, total amount (if meaningful), account count, trigger summary, history coverage and status.

• Filtering: priority, open/reviewed state, scenario/source and signal type. Sorting should be deterministic.

• Do not display “fraudster” or “guilty”; use “potentially suspicious”, “requires review” or “case priority”.

Screen B: case investigation

• A graph showing only the nodes/edges relevant to the case, with event IDs in tooltips and a legend for source/destination/direction.

• A chronological transaction table with amount, step/time, source, destination, and rule/feature markers.

• Evidence panel containing each trigger, measured value, threshold/configuration, model score, coverage and limitations.

• Reviewer controls: status, notes, and action history. These are local demo workflow records, not bank actions.

Screen C: evaluation

• Dataset name/type, source, date downloaded, rows, label count/prevalence, split boundaries and feature list.

• AP/PR chart and metric cards with counts. Include threshold selection and confusion matrix.

• Scenario comparison: D01 suspicious chain vs D02 legitimate high activity vs D03 insufficient history.

Error handling and edge cases

Input problem	Expected handling
Negative or non-numeric amount	Reject row with explicit validation error; do not silently coerce.
Missing account ID	Reject row or flag as unscorable under a documented import policy.
Out-of-order events	Sort by timestamp with stable tie-breaker; record that ordering was normalised.
Duplicate transaction ID	Reject duplicates or use an explicit de-duplication rule; never silently double count.
New account / little history	Mark limited coverage; suppress personal-baseline features until sufficient observations exist.
No positive labels in a test segment	Do not report AP as a meaningful model comparison for that split; revise split with transparent reasoning.

14. Security, governance and ethical boundaries

Although TraceGuard is a hackathon prototype, it should not handle real bank credentials, real account exports or personal financial records. Financial fraud scores can have consequences; a false alert is not a harmless UI event.

Minimum safeguards

• Use seeded/synthetic IDs, no real customer PII and no secrets in Git.

• Keep model version, feature version, dataset/scenario ID, score components, rule versions and reviewer actions in an audit record.

• Separate benchmark labels from inference payloads; do not expose labels in the UI before review is completed.

• Use least-privilege local configuration and avoid unauthenticated external exposure. For a local demo, localhost is preferable to public deployment.

• Mark synthetic, benchmark and production data modes clearly. Log enough for reproducibility but avoid storing sensitive raw records.

• Explain that alerts are decision support; a human reviews cases. No automatic freeze, block or adverse action is performed.

RBI FREE-AI principles translated into build tasks

Principle theme	TraceGuard implementation action
Trust / accountability	Keep dataset provenance, model versions, configuration and decision audit trail.
People first / fairness	Test legitimate high-volume and new-account scenarios; avoid guilt language.
Explainability	Show raw evidence, calculated signals and feature attribution where available.
Safety / resilience	Offline deterministic demo, robust input validation and clear failure states.
Governance	Document who/what produced a score, its limits and the required human review.

The above translates the RBI report into a proposed student-project checklist; it is not a legal determination that the prototype complies with every financial-sector requirement. **[S20]**

Related incident response links

For real incidents in India, use official reporting channels and the relevant financial institution. The National Cyber Crime Reporting Portal is at cybercrime.gov.in. TraceGuard must not promise that it can recover funds or replace official complaint procedures. **[S22]**

15. Remaining hackathon schedule (24-26 October)

The official event listing gives a start time of 09:30 IST on Saturday 24 October 2026. This report is dated 24 October 2026 and assumes work begins from approximately 12:30 IST; if your actual team timer differs, shift the blocks but keep the checkpoint outputs. The official format says progress is evaluated every eight hours. **[S01]**

Window (IST)	Main objective	Exit evidence
Sat 24 Oct, 12:30-17:30	Lock scope; repo; demo fixtures; backend/ frontend skeleton; first vertical slice.	Scenario loads and a rule-based case appears in UI. Ready for first 8-hour checkpoint.
Sat 24 Oct, 17:30-Sun 01:30	PaySim validation + chronological split; baseline metrics; API import; case timeline.	Reproducible baseline report; no label/flag leakage.

Window (IST)	Main objective	Exit evidence
Sun 25 Oct, 01:30-09:30	Graph features and sequence grouping; queue sorting; review state.	D01 suspicious chain and D02 legitimate burst work offline.
Sun 25 Oct, 09:30-17:30	One stronger model, validation-selected threshold and explainable evidence.	Evaluation screen with AP/precision/recall/FP and documented split.
Sun 25 Oct, 17:30-Mon 26 Oct, 01:30	Robustness, integration fixes, model card, data provenance, README.	Fresh start command and core tests pass.
Mon 26 Oct, 01:30-09:30	Freeze features; rehearse 60-second demo; final clean clone and recording.	Stable submission, links/docs, backup demo and no unresolved P0 bug.

Checkpoint strategy

• Every 8 hours, demonstrate the working product and record what changed. Do not wait until the end to reveal an unintegrated model.

• By the first checkpoint, a rule-based end-to-end path should work even if the dataset/model is not ready.

• At the final checkpoint, stop adding features. Fix reproducibility and demo reliability.

Team role allocation

Role	Owns	Must coordinate with
Data/ML	Dataset validation, feature pipeline, train/validation/test split, metrics, model artifact.	Backend for feature schema and frontend for displayed metrics.
Backend/Graph	API, graph features, case grouping, persistence, review/audit state.	Data/ML on feature contract; frontend on endpoints.
Frontend	Queue, case details, graph/timeline, scenario and evaluation screens.	Backend on response schema; QA on interactions.
Integration/QA (4th member)	Seed fixtures, test matrix, fresh setup, docs, demo timing and fallback.	Everyone; protect integration time.

For a two-person team, combine data/model with backend and assign the second person frontend plus QA. Reduce model variants to one baseline and one optional boosted tree model.

16. One-minute demo and likely judge questions

Recommended 60-second script

Time	Action	Narration (example)
0-8s	Open investigation queue.	“TraceGuard prioritises suspicious sequences, not only high-value individual transfers.”
8-23s	Open D01 and point to transaction path.	“This case links an incoming-to-outgoing sequence across these accounts. Here are the exact event IDs, timing and calculated signals.”
23-35s	Open evidence panel.	“The model score, rule evidence and graph signal are shown separately. The score is a review priority, not a verdict.”
35-45s	Replay D02 legitimate burst.	“High volume alone is not enough; the benign case shows the system’s false-positive challenge.”
45-54s	Replay modified input or D03.	“Changing the timing recalculates the sequence signal. With little history, the system surfaces uncertainty.”
54-60s	Show evaluation summary.	“These metrics use a chronological holdout on synthetic PaySim data. They are not claims of performance on real Indian UPI data.”

Likely questions and defensible answers

Q1. What is new?

“Graph analysis and fraud models already exist. Our focus is a complete evidence-first case workflow combining independently visible model, rule and network signals, limited-history handling, legitimate-control scenarios and reproducible evaluation.”

Q2. Is this trained on real UPI transactions?

“No. The baseline uses PaySim synthetic mobile-money data and the live demo uses team-generated synthetic scenarios. We do not claim a real Indian UPI dataset or bank integration.”

Q3. Why not use accuracy?

“Fraud labels are rare in PaySim; an all-negative predictor can achieve very high accuracy while detecting zero fraud labels. We report AP, precision, recall, false-positive counts and precision at a stated alert budget.” **[S08]** **[S16]**

Q4. Does a high score mean a person is guilty?

“No. It means the case is prioritised for review under the prototype’s model and policy. We display evidence and limitations and require human review.”

Q5. How do you prevent data leakage?

“The label is evaluation-only; the existing `isFlaggedFraud` field is excluded from model features; the split is chronological; and every feature is computed using permitted history as of the scoring event. Preprocessing is fitted on training data only.” **[S17]**

Q6. Can it stop a fraudulent payment?

“No. It is a retrospective investigation prototype on public/synthetic data. Payment blocking requires authorised institutional integration and controls. Our scope is detection, prioritisation and explanation.”

Q7. How would it work in production?

“A production deployment would require authorised data access, privacy/security and governance review, operational threshold calibration, real-world validation, monitoring/drift controls, incident response and approval by the institution. Our prototype does not establish those readiness conditions.”

17. Coding-AI handoff: implementation prompt

Copy the prompt below into the coding AI working on the repository. The linked sources are reference material; the AI should inspect the current repository and package versions rather than blindly copy stale examples.

Master implementation prompt

Act as the senior engineer for TraceGuard, a 48-hour student hackathon prototype for Srujana Finance PS-1 (Financial Fraud and Transaction Risk Detection). Build a working, testable and explainable transaction investigation workbench. Do not claim to connect to real UPI/bank data or the I4C Suspect Registry.

Priority order: (1) working seeded demo end-to-end, (2) validated CSV ingestion, (3) transparent rule/sequence/graph evidence, (4) investigator case queue and review status, (5) reproducible chronological PaySim baseline, (6) optional stronger tree model and SHAP only if stable.

Architecture: React/Vite frontend; FastAPI backend; Python feature/model modules; NetworkX for small graph analysis; SQLite unless PostgreSQL is already working; scikit-learn baseline; XGBoost only if it installs quickly.

Data rules: use PaySim only after checking its current source/terms and validating the file; document that it is synthetic. Keep a small hand-verified synthetic demo stream. Never use `isFraud` as a feature; exclude `isFlaggedFraud`; use chronological splitting; compute rolling features using only past events plus permitted current-event fields. Make scoring time explicit. Do not invent location/device fields for PaySim.

UI workflow: queue -> case detail -> connected account graph/timeline -> evidence panel -> review status -> evaluation page. Include three deterministic scenarios: suspicious pass-through chain, legitimate high-volume merchant/payroll burst, and new account with insufficient history. The app must run locally without a network call after setup.

Evaluation: compare a rule baseline with at most one stronger model on identical chronological splits. Report average precision, precision, recall, false-positive counts, confusion counts and precision@K. Do not claim high performance before measuring it. Do not tune on the final test split.

Engineering quality: use type/schema validation, deterministic seeds, small pure functions, automated tests, clear API errors, no secrets or personal bank data, reproducible startup, dependency versions, README, data provenance, model card, limitations and one-minute demo script.

Workflow: inspect current files first; create a task checklist; implement P0 requirements before P1; run tests after each integration; never replace a working path with an unfinished advanced feature. If a dependency or model fails, preserve the rules/graph demo and label the fallback honestly. Explain all assumptions and never invent benchmark metrics.

A useful source inspection sequence for a coding AI

1. Read the Srujana hackathon instructions and the supplied rubric before implementation. Confirm the final deliverable format and repo naming. **[S01]**
2. Read the PaySim paper and dataset description, then validate the actual file and terms locally. **[S07] [S08] [S09]**
3. Read the 2026 CERT-In advisory and MHA/RBIH release to understand current context, but do not turn them into unsupported detection rules without relevant data. **[S03] [S05]**
4. Study AMLSim and Neo4j examples only for ideas. Do not add their dependencies until the core app runs. **[S10] [S11] [S12]**
5. Use scikit-learn metric and leakage documentation when implementing evaluation. Check XGBoost and SHAP docs if those libraries are used. **[S16] [S17] [S18] [S19]**
6. Run clean-start instructions and all tests; then rehearse the exact 60-second demo.

Repository acceptance checklist

• README includes overview, setup, architecture, endpoints, sample workflow, dataset source/limitations, evaluation command and demo script.

• `docs/data_provenance.md` lists source, retrieval date, schema, row/label counts and hash.

• `docs/model_card.md` lists intended use, excluded uses, data, split, metrics, limitations and threshold method.

• Tests verify target/flag exclusion, chronological split, no-history behavior, graph path evidence and deterministic replay.

• No secrets, bank credentials, personal data, dataset files prohibited by source terms, or undocumented third-party assets are committed.

18. Final go/no-go checklist

Gate	Go only when...	If not met
Demo gate	D01, D02 and D03 work locally; core path has no network dependency.	Cut model/extra screens; fix the vertical slice.
Data gate	Dataset source/terms/schema and synthetic status are documented.	Use only hand-verified fixtures until data source is verified.
Model gate	Metrics are reproducible, split is chronological and leakage checks pass.	Show rule/graph baseline only; never fake metrics.
Evidence gate	Every alert reason points to raw events and computed features.	Remove generic explanation text that is not tied to evidence.
Correctness gate	Benign and insufficient-history scenarios behave as intended.	Do not claim production suitability.
Submission gate	GitHub repo follows naming pattern; README and demo video are prepared.	Freeze features and finish submission packaging.

Hard truth

A complex model with unreliable validation is weaker than a smaller working investigation product that shows its evidence, passes edge cases and clearly communicates what it cannot know. The aim is not “perfect fraud detection”; no credible 48-hour prototype can establish that. The aim is a defensible, testable solution to the PS-1 workflow.

Final build order

1. Scenario replay + rules + queue. 2. Case graph and evidence. 3. Input validation and audit/review. 4. PaySim benchmark pipeline. 5. Correct chronological metrics. 6. Optional tree model/explanation. 7. Demo polish, documentation and clean-start test. This order protects the event’s most important output: a one-minute working demo backed by a credible repository.

19. Verified source library

Every linked title below opens the source page. Official sources are used for current institutional claims; research and repositories are used for methodology/prior art. Source state and displayed statistics are as of 24 October 2026 where noted. Re-check dynamic pages before final submission.

Important: repository presence does not imply code quality, licensing compatibility or relevance to the current environment. Verify the exact branch, file, licence and data provenance before copying code.

S01 - Srujana Tech Fest 2026 - official hackathon page
Official event details, schedule, submission requirements and one-minute demo emphasis. Accessed 24 October 2026.
https://www.srujanacu.in/events/hackathon

S02 - Srujana Tech Fest 2026 - Participant Rubric and Submission Guide
User-supplied PDF, 3 pages. Used for the seven weighted judging criteria and rules on data, dependencies, testing and limitations.
Source: supplied participant rubric PDF (pages 1-3); not a public external URL.

S03 - MHA / PIB: I4C and RBIH sign MoU for AI-driven detection of mule accounts
Official Government of India release, 12 May 2026. Context for mule-account intelligence sharing and AI-assisted risk assessment.
https://www.pib.gov.in/PressReleasePage.aspx?PRID=2260277□=1®=3

S04 - MHA / PIB: “Misleading Mobile Applications” release with CFCFRMS and Suspect Registry statistics
Official Government of India release, 11 August 2026. Reports CFCFRMS and Suspect Registry totals as of 30 June 2026; these are system-level reported figures, not TraceGuard performance.
https://www.pib.gov.in/PressReleasePage.aspx?PRID=2297626□=1®=3

S05 - CERT-In: Advisory CIAD-2026-0046 - payment and disbursement API abuse
Official CERT-In advisory, original issue date 9 October 2026. Describes unauthorized fund transfers through compromised or abused payment/disbursement APIs, including transfers to mule accounts.
https://www.cert-in.org.in/s2cMainServlet?VLCODE=CIAD-2026-0046&pageid;=PUBVLNOTES02

S06 - NPCI: UPI product statistics
Official NPCI statistics page. The current page displayed September 2026 volume/value figures at research time; figures may be updated by NPCI.
https://www.npci.org.in/product/upi/product-statistics

S07 - Lopez-Rojas, Elmir and Axelsson: PaySim, a financial mobile money simulator for fraud detection
2016 conference proceedings page describing the PaySim synthetic mobile-money simulator and the shortage of public real transaction datasets.
https://www.msc-les.org/proceedings/emss/emss2016/emss2016_249.html

S08 - Follow the Trail: Machine Learning for Fraud Detection in Fintech Applications
Open-access research article describing PaySim data characteristics, including 6,362,620 transactions and the feature schema; treat dataset properties as benchmark-specific.
https://pmc.ncbi.nlm.nih.gov/articles/PMC7956727/

S09 - PaySim1 dataset page on Kaggle
Public dataset landing page. Download access, account requirements and terms may change; confirm provenance and permitted use before relying on it.
https://www.kaggle.com/ntnu-testimon/paysim1

S10 - IBM AMLSim - synthetic anti-money-laundering transaction simulator
Open-source repository describing a simulator that produces synthetic banking transactions and known laundering patterns. README lists Apache-2.0 licence and legacy dependencies; inspect before use.
https://github.com/IBM/AMLSim

S11 - Neo4j graph examples: fraud detection with PaySim

Existing graph-based example using PaySim and Neo4j Graph Data Science/Bloom. Repository describes compatibility with older Neo4j versions; treat as a reference, not an assumed drop-in dependency.

https://github.com/neo4j-graph-examples/fraud-detection

S12 - Neo4j product example: graph-native ML on PaySim

Existing money-mule detection demo using synthetic PaySim network data. README mentions a Neo4j AuraDS/Enterprise environment; not required for the proposed stack.

https://github.com/neo4j-product-examples/ml-paysim-fraud-detection-demo

S13 - Public UPI Fraud Detection Using Machine Learning repository

Example of an existing UPI-labelled project with an app and notebook. Its existence proves the idea is not new; its page alone does not establish that its data are real Indian UPI records.

https://github.com/Vatshayan/UPI-Fraud-Detection-Using-Machine-Learning

S14 - Scientific Reports (2026): A robust machine learning framework for detecting temporal drift in financial fraud prevention

Open-access article published 18 June 2026. Abstract describes extreme class imbalance, temporal non-stationarity and the unequal costs of missed fraud and false alarms; data context differs from Indian UPI.

https://www.nature.com/articles/s41598-026-58285-5

S15 - Scientific Reports (2026): A drift adaptive framework for detecting and tracking evolving anomalies in financial transaction streams

Recent article listed as published 6 September 2026. Useful as context for evolving transaction behaviour; not evidence that any specific technique will work for TraceGuard without testing.

https://www.nature.com/articles/s41598-026-70457-x

S16 - scikit-learn: average_precision_score documentation

Current official metric documentation. Defines Average Precision and distinguishes it from trapezoidal PR-area calculations.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

S17 - scikit-learn: common pitfalls and data leakage

Official documentation explaining leakage and why learned preprocessing must be fitted only on training data.

https://scikit-learn.org/stable/common_pitfalls.html

S18 - XGBoost: parameter reference

Official parameter docs; documents scale_pos_weight for imbalanced classes. This is a tuning option, not a guarantee of better results.

https://xgboost.readthedocs.io/en/stable/parameter.html

S19 - SHAP: TreeExplainer documentation

Official docs for explaining ensemble-tree outputs. Explanation values are model-output attributions, not proof of causality or guilt.

https://shap.readthedocs.io/en/latest/generated/shap.TreeExplainer.html

S20 - RBI: FREE-AI Committee Report - Responsible and Ethical Enablement of AI

Official RBI report page, dated 13 August 2025. Sets out principles including trust, people first, fairness, accountability, explainability and safety/resilience.

https://m.rbi.org.in/scripts/PublicationReportDetails.aspx?ID=1306&UrlPage;=

S21 - CERT-In: cyber security advisories

CERT-In advisory portal and the directly linked 9 October 2026 payment/disbursement API advisory.

https://www.cert-in.org.in/s2cMainServlet?VLCODE=CIAD-2026-0046&pageid;=PUBVLNOTES02

S22 - National Cyber Crime Reporting Portal

Official portal for reporting cybercrime. This project must not claim to replace official reporting or incident-response workflows.

https://cybercrime.gov.in/

Prepared 24 October 2026. This is a research and implementation planning document, not a guarantee of hackathon placement, financial accuracy in production, a fraud determination, or legal/regulatory advice. External references should be re-opened to confirm current content and applicable terms.